

CovidDB Use Cases

Joins

1. Which country has the highest number of confirmed cases on a specific date?

Query	Query History		
1 2 3 4 5 6 7 8 9 10	<pre>SELECT c.name AS country, g.report_date, g.confirmed FROM global_covid_stats g JOIN country c ON g.country_id = c.country_id WHERE g.report_date = '2020-09-30' ORDER BY g.confirmed DESC LIMIT 1;</pre>		
	country character varying (100)	report_date date	confirmed bigint
	1	India	2020-09-30
			11500036

2. Show the total number of deaths in each country, including provinces/states, for a given date.

Query	Query History																																																					
1	SELECT																																																					
2	c.name AS country,																																																					
3	s.name AS state,																																																					
4	cs.report_date,																																																					
5	SUM(cs.deaths) AS total_deaths																																																					
6	FROM covid_case_stats cs																																																					
7	JOIN country c																																																					
8	ON cs.country_id = c.country_id																																																					
9	JOIN state s																																																					
10	ON cs.state_id = s.state_id																																																					
11	WHERE cs.report_date = '2020-09-30'																																																					
12	GROUP BY																																																					
13	c.name,																																																					
14	s.name,																																																					
15	cs.report_date																																																					
16	ORDER BY total_deaths DESC;																																																					
	<table><tr><th>country</th><th>state</th><th>report_date</th><th>total_deaths</th></tr><tr><td>character varying (100)</td><td>character varying (100)</td><td>date</td><td>bigint</td></tr><tr><td>1</td><td>India</td><td>Maharashtra</td><td>2020-09-30</td><td>36181</td></tr><tr><td>2</td><td>India</td><td>Tamil Nadu</td><td>2020-09-30</td><td>9453</td></tr><tr><td>3</td><td>India</td><td>Karnataka</td><td>2020-09-30</td><td>8777</td></tr><tr><td>4</td><td>India</td><td>Andhra Pradesh</td><td>2020-09-30</td><td>5780</td></tr><tr><td>5</td><td>India</td><td>Uttar Pradesh</td><td>2020-09-30</td><td>5715</td></tr><tr><td>6</td><td>India</td><td>Delhi</td><td>2020-09-30</td><td>5320</td></tr><tr><td>7</td><td>India</td><td>West Bengal</td><td>2020-09-30</td><td>4899</td></tr><tr><td>8</td><td>India</td><td>Gujarat</td><td>2020-09-30</td><td>3439</td></tr><tr><td>9</td><td>India</td><td>Punjab</td><td>2020-09-30</td><td>3359</td></tr></table>	country	state	report_date	total_deaths	character varying (100)	character varying (100)	date	bigint	1	India	Maharashtra	2020-09-30	36181	2	India	Tamil Nadu	2020-09-30	9453	3	India	Karnataka	2020-09-30	8777	4	India	Andhra Pradesh	2020-09-30	5780	5	India	Uttar Pradesh	2020-09-30	5715	6	India	Delhi	2020-09-30	5320	7	India	West Bengal	2020-09-30	4899	8	India	Gujarat	2020-09-30	3439	9	India	Punjab	2020-09-30	3359
country	state	report_date	total_deaths																																																			
character varying (100)	character varying (100)	date	bigint																																																			
1	India	Maharashtra	2020-09-30	36181																																																		
2	India	Tamil Nadu	2020-09-30	9453																																																		
3	India	Karnataka	2020-09-30	8777																																																		
4	India	Andhra Pradesh	2020-09-30	5780																																																		
5	India	Uttar Pradesh	2020-09-30	5715																																																		
6	India	Delhi	2020-09-30	5320																																																		
7	India	West Bengal	2020-09-30	4899																																																		
8	India	Gujarat	2020-09-30	3439																																																		
9	India	Punjab	2020-09-30	3359																																																		

3. List the continents along with the total number of confirmed cases, deaths, and recoveries.

Query	Query History		continent character varying (50)	total_confirmed_cases numeric	total_deaths numeric	total_recoveries numeric
1	SELECT					
2	c.continent,					
3	SUM(g.confirmed) AS total_confirmed_cases,	1	Asia	163130042	2936323	153342237
4	SUM(g.deaths) AS total_deaths,	2	North America	43546773	783829	40933962
5	SUM(g.recovered) AS total_recoveries	3	Europe	41133247	740378	38665244
6	FROM global_covid_stats g	4	Africa	32180975	579248	30250108
7	JOIN country c	5	South America	27005824	486094	25385465
8	ON g.country_id = c.country_id	6	Oceania	2231238	40157	2097363
9	GROUP BY c.continent					
10	ORDER BY total_confirmed_cases DESC ;					

Aggregate Functions

4. Calculate the average number of new deaths per day across all countries.

Query	Query History		report_date date	average_new_deaths numeric
1	SELECT	1	2020-06-30	9938.9500000000000000
2	report_date,	2	2020-09-30	11926.80000000000000
3	AVG(new_deaths) AS average_new_deaths	3	2020-12-31	13914.85000000000000
4	FROM global_covid_stats	4	2021-03-31	9938.9500000000000000
5	GROUP BY report_date	5	2021-06-30	11926.80000000000000
6	ORDER BY report_date;	6	2021-09-30	13914.85000000000000

5. Find the maximum number of active cases recorded in any country on a specific date.

Query	Query History		country character varying (100)	report_date date	active_cases bigint
1	SELECT				
2	c.name AS country,				
3	g.report_date,				
4	g.active_cases				
5	FROM global_covid_stats g				
6	JOIN country c				
7	ON g.country_id = c.country_id				
8	WHERE g.report_date = '2020-06-30'				
9	ORDER BY g.active_cases DESC				
10	LIMIT 1;				
		1	India	2020-06-30	241501

Stored Procedure

6. Create a stored procedure that returns the total number of recovered cases for a given country and date.

Query	Query History									
<pre>1 CREATE OR REPLACE FUNCTION get_recovered_cases(2 p_country_name VARCHAR, 3 p_date DATE 4) 5 RETURNS BIGINT 6 LANGUAGE plpgsql 7 AS \$\$ 8 DECLARE 9 total_recovered BIGINT; 10 BEGIN 11 SELECT COALESCE(SUM(g.recovered), 0) 12 INTO total_recovered 13 FROM global_covid_stats g 14 JOIN country c 15 ON g.country_id = c.country_id 16 WHERE c.name = p_country_name 17 AND g.report_date = p_date; 18 19 RETURN total_recovered; 20 END; 21 \$\$; 22 SELECT get_recovered_cases('India', '2020-09-30');</pre>	<table border="1"><thead><tr><th></th><th>get_recovered_cases</th><th></th></tr></thead><tbody><tr><td></td><td>bigint</td><td></td></tr><tr><td>1</td><td></td><td>10810034</td></tr></tbody></table>		get_recovered_cases			bigint		1		10810034
	get_recovered_cases									
	bigint									
1		10810034								

7. Design a stored procedure to update the number of deaths for a specific country and date.

Query	Query History
<pre>1 CREATE OR REPLACE PROCEDURE update_deaths(2 p_country_name VARCHAR, 3 p_date DATE, 4 p_new_deaths BIGINT 5) 6 LANGUAGE plpgsql 7 AS \$\$ 8 BEGIN 9 UPDATE global_covid_stats g 10 SET deaths = p_new_deaths 11 FROM country c 12 WHERE g.country_id = c.country_id 13 AND c.name = p_country_name 14 AND g.report_date = p_date; 15 END; 16 \$\$; 17 CALL update_deaths(18 'India', 19 '2020-09-30', 20 500000 21);</pre>	

Views

8. Create a view that displays the total number of cases (confirmed, deaths, and recovered) for each country on a specific date.

Query	Query History		country	report_date	confirmed	deaths	recovered
			character varying (100)	date	numeric	numeric	numeric
1	CREATE OR REPLACE VIEW country_covid_summary AS						
2	SELECT						
3	c.name AS country,	1	India	2020-09-30	11500036	500000	10810034
4	g.report_date,						
5	SUM(g.confirmed) AS confirmed,	2	United States	2020-09-30	2758355	49650	2592854
6	SUM(g.deaths) AS deaths,						
7	SUM(g.recovered) AS recovered	3	Indonesia	2020-09-30	2279363	41028	2142601
8	FROM global_covid_stats g						
9	JOIN country c	4	Brazil	2020-09-30	1771328	31883	1665048
10	ON g.country_id = c.country_id						
11	GROUP BY	5	Nigeria	2020-09-30	1717829	30920	1614759
12	c.name,						
13	g.report_date;	6	Russia	2020-09-30	1216120	21890	1143153
14							
15	SELECT *	7	Mexico	2020-09-30	1074439	19339	1009972
16	FROM country_covid_summary						
17	WHERE report_date = '2020-09-30'	8	Japan	2020-09-30	1053970	18971	990732
18	ORDER BY confirmed DESC;						
		9	Egypt	2020-09-30	852786	15250	801610

9. Implement a view to show the latest data (confirmed, deaths, recovered) for each country.

Query	Query History		country	report_date	confirmed	deaths	recovered
			character varying (100)	date	bigint	bigint	bigint
1	CREATE OR REPLACE VIEW latest_country_covid_data AS						
2	SELECT						
3	c.name AS country,	1	India	2021-09-30	34500109	621001	32430102
4	g.report_date,						
5	g.confirmed,	2	United States	2021-09-30	8275066	148951	7778562
6	g.deaths,						
7	g.recovered	3	Indonesia	2021-09-30	6838090	123085	6427804
8	FROM global_covid_stats g						
9	JOIN country c	4	Brazil	2021-09-30	5313985	95651	4995145
10	ON g.country_id = c.country_id						
11	WHERE g.report_date = (	5	Nigeria	2021-09-30	5153489	92762	4844279
12	SELECT MAX(report_date)						
13	FROM global_covid_stats	6	Russia	2021-09-30	3648361	65670	3429459
14	);						
15	SELECT *	7	Mexico	2021-09-30	3223318	58019	3029918
16	FROM latest_country_covid_data						
17	ORDER BY confirmed DESC;	8	Japan	2021-09-30	3161911	56914	2972196
		9	Egypt	2021-09-30	852786	15250	801610

T-SQL

10. Write a T-SQL query to calculate the total number of cases (confirmed + deaths + recovered) for each country.

Query	Query History		country character varying (100)	total_cases numeric
1	SELECT			
2	c.name AS country,	1	India	236722237
3	SUM(	2	United States	56709022
4	g.confirmed	3	Indonesia	46861420
5	+ g.deaths	4	Brazil	36416726
6	+ g.recovered	5	Nigeria	35316848
7	) AS total_cases	6	Russia	25002211
8	FROM global_covid_stats g	7	Mexico	22089386
9	JOIN country c	8	Japan	21668569
10	ON g.country_id = c.country_id			
11	GROUP BY c.name			
12	ORDER BY total_cases DESC;			

11. Use T-SQL to identify the country with the highest number of new cases reported on a specific date.

Query	Query History		country character varying (100)	report_date date	new_confirmed bigint
1	SELECT				
2	c.name AS country,				
3	g.report_date,				
4	g.new_confirmed				
5	FROM global_covid_stats g				
6	JOIN country c				
7	ON g.country_id = c.country_id				
8	WHERE g.report_date = '2020-09-30'				
9	ORDER BY g.new_confirmed DESC				
10	LIMIT 1;	1	India	2020-09-30	6210019

CTE (Common Table Expressions)

12. Create a CTE to calculate the percentage increase in confirmed cases for each country over the past week.

```
Query  Query History
1  WITH weekly_cases AS (
2      SELECT
3          c.name AS country,
4          g.report_date,
5          g.confirmed,
6          LAG(g.confirmed, 7) OVER (
7              PARTITION BY g.country_id
8              ORDER BY g.report_date
9          ) AS previous_week_confirmed
10     FROM global_covid_stats g
11     JOIN country c
12         ON g.country_id = c.country_id
13 )
14 SELECT
15     country,
16     report_date,
17     confirmed,
18     previous_week_confirmed,
19     ROUND(
20         (
21             (confirmed - previous_week_confirmed) * 100
22             / NULLIF(previous_week_confirmed, 0)
23         )::NUMERIC,
24         2
25     ) AS percentage_increase
26 FROM weekly_cases
27 WHERE previous_week_confirmed IS NOT NULL
28 ORDER BY percentage_increase DESC;
```

13. Use a CTE to find the country with the highest number of active cases at the moment.

```
Query  Query History
1  WITH latest_data AS (
2      SELECT
3          g.country_id,
4          g.active_cases,
5          g.report_date
6      FROM global_covid_stats g
7      WHERE g.report_date = (
8          SELECT MAX(report_date)
9          FROM global_covid_stats
10     )
11 )
12 SELECT
13     c.name AS country,
14     l.report_date,
15     l.active_cases
16 FROM latest_data l
17 JOIN country c
18     ON l.country_id = c.country_id
19 ORDER BY l.active_cases DESC
20 LIMIT 1;
```

	country character varying(100)	report_date date	active_cases bigint
1	India	2021-09-30	1449006

Indexes

14. Explain the importance of indexes in optimizing queries for this dataset.

Query	Query History	QUERY PLAN
		text
1	EXPLAIN ANALYZE	1 Seq Scan on global_covid_stats (cost=0.00..3.50 rows=6 width=76) (actual time=0.023..0.028 rows=6.00 loop...
2	SELECT *	2 Filter: (country_id = 1)
3	FROM global_covid_stats	3 Rows Removed by Filter: 114
4	WHERE country_id = 1;	4 Buffers: shared hit=2
		5 Planning:
		6 Buffers: shared hit=6 dirtied=1
		7 Planning Time: 1.368 ms
		8 Execution Time: 0.054 ms

15. Implement an index on the "Country/Region" column to speed up search operations.

Query	Query History
1	CREATE INDEX idx_country_name
2	ON country(name);
3	SELECT
4	indexname,
5	indexdef
6	FROM pg_indexes
7	WHERE tablename = 'country';
8	EXPLAIN ANALYZE
9	SELECT *
10	FROM country
11	WHERE name = 'India';

	indexname	indexdef
	name	text
1	country_pkey	CREATE UNIQUE INDEX country_pkey ON public.country USING btree (country...
2	country_name_...	CREATE UNIQUE INDEX country_name_key ON public.country USING btree (n...
3	idx_country_na...	CREATE INDEX idx_country_name ON public.country USING btree (name)

	QUERY PLAN
	text
1	Seq Scan on country (cost=0.00..1.25 rows=1 width=348) (actual time=0.027..0.030 rows=1.00 loop...
2	Filter: ((name)::text = 'India'::text)
3	Rows Removed by Filter: 19
4	Buffers: shared hit=1
5	Planning:
6	Buffers: shared hit=12
7	Planning Time: 2.253 ms
8	Execution Time: 0.061 ms

User-Defined Functions (UDF)

16. Develop a UDF to calculate the mortality rate (deaths / confirmed cases * 100) for a given country.

Query	Query History		
1	CREATE OR REPLACE FUNCTION calculate_mortality_rate(2 p_country_name VARCHAR 3) 4 RETURNS NUMERIC 5 LANGUAGE plpgsql 6 AS \$\$ 7 DECLARE 8 mortality_rate NUMERIC; 9 BEGIN 10 SELECT 11 (12 SUM(g.deaths) * 100.0 13 / NULLIF(SUM(g.confirmed), 0) 14) 15 INTO mortality_rate 16 FROM global_covid_stats g 17 JOIN country c 18 ON g.country_id = c.country_id 19 WHERE c.name = p_country_name; 20 21 RETURN ROUND(mortality_rate, 2); 22 END; 23 \$\$; 24 SELECT calculate_mortality_rate('India');		
		calculate_mortality_rate	numeric
		1	2.04

17. Create a UDF to determine the recovery rate (recovered / confirmed cases * 100) for a specific date.

Query	Query History		
1	CREATE OR REPLACE FUNCTION calculate_recovery_rate(2 p_country_name VARCHAR, 3 p_date DATE 4) 5 RETURNS NUMERIC 6 LANGUAGE plpgsql 7 AS \$\$ 8 DECLARE 9 recovery_rate NUMERIC; 10 BEGIN 11 SELECT 12 (13 SUM(g.recovered) * 100.0 14 / NULLIF(SUM(g.confirmed), 0) 15) 16 INTO recovery_rate 17 FROM global_covid_stats g 18 JOIN country c 19 ON g.country_id = c.country_id 20 WHERE c.name = p_country_name 21 AND g.report_date = p_date; 22 23 RETURN ROUND(recovery_rate, 2); 24 END; 25 \$\$; 26 SELECT calculate_recovery_rate(27 'India', 28 '2020-09-30' 29);		
		calculate_recovery_rate	numeric
		1	94.00

Group By

18. Group the data by continent and calculate the total number of confirmed cases for each continent.

Query	Query History		continent character varying (50)	total_confirmed_cases numeric
1	SELECT			
2	c.continent,	1	Asia	163130042
3	SUM(g.confirmed) AS total_confirmed_cases	2	North America	43546773
4	FROM global_covid_stats g	3	Europe	41133247
5	JOIN country c	4	Africa	32180975
6	ON g.country_id = c.country_id	5	South America	27005824
7	GROUP BY c.continent	6	Oceania	2231238
8	ORDER BY total_confirmed_cases DESC;			

19. Group the data by date and compute the total number of deaths and recoveries for each date.

Query	Query History		report_date date	total_deaths numeric	total_recoveries numeric
1	SELECT	1	2020-06-30	265043	13841630
2	g.report_date,	2	2020-09-30	823095	27683271
3	SUM(g.deaths) AS total_deaths,	3	2020-12-31	795147	41524912
4	SUM(g.recovered) AS total_recoveries	4	2021-03-31	1060196	55366547
5	FROM global_covid_stats g	5	2021-06-30	1325244	69208186
6	GROUP BY g.report_date	6	2021-09-30	1590304	83049833
7	ORDER BY g.report_date;				

20. Group the data by country and calculate the average number of new cases reported daily for each country.

Query	Query History		country character varying (100)	average_daily_new_cases numeric
1	SELECT	1	India	6210019.000000000000
2	c.name AS country,	2	United States	1489511.000000000000
3	AVG(g.new_confirmed) AS average_daily_new_cases	3	Indonesia	1230855.666666666667
4	FROM global_covid_stats g	4	Brazil	956516.666666666667
5	JOIN country c	5	Nigeria	927627.666666666667
6	ON g.country_id = c.country_id	6	Russia	656704.333333333333
7	GROUP BY c.name	7	Mexico	580196.666666666667
8	ORDER BY average_daily_new_cases DESC;	8	Japan	569143.333333333333
		9	Egypt	460504.333333333333

COVID-Data-Global

21. To find out the death percentage locally and globally.

Global

Query	Query History		global_death_percentage numeric
1	SELECT		
2	ROUND(		
3	(		
4	SUM(deaths) * 100.0		
5	/ NULLIF(SUM(confirmed), 0)		
6	::NUMERIC,		
7	2		
8	) AS global_death_percentage		
9	FROM global_covid_stats;	1	1.89

Country-wise

Query	Query History		country character varying (100)	death_percentage numeric
1	SELECT			
2	c.name AS country,	1	Colombia	1.80
3	ROUND(	2	United States	1.80
4	(	3	Turkey	1.80
5	MAX(g.deaths) * 100.0	4	Nigeria	1.80
6	/ NULLIF(MAX(g.confirmed), 0)	5	Spain	1.80
7	)::NUMERIC,	6	Argentina	1.80
8	2	7	Indonesia	1.80
9	) AS death_percentage	8	Italy	1.80
10	FROM global_covid_stats g	9	Egypt	1.80
11	JOIN country c			
12	ON g.country_id = c.country_id			
13	GROUP BY c.name			
14	ORDER BY death_percentage DESC ;			

22. To find out the infected population percentage locally and globally

Global

Query	Query History		global_infected_population_percentage numeric
1	SELECT		
2	ROUND(		
3	(		
4	SUM(g.confirmed) * 100.0		
5	/ NULLIF(SUM(c.population), 0)		
6	)::NUMERIC,		
7	2		
8	) AS global_infected_population_percentage		
9	FROM global_covid_stats g		
10	JOIN country c		
11	ON g.country_id = c.country_id;		
		1	1.46

Country-wise

Query	Query History
1	SELECT
2	c.name AS country,
3	c.population,
4	MAX (g.confirmed) AS confirmed_cases,
5	ROUND (
6	(
7	MAX (g.confirmed) * 100.0
8	/ NULLIF (c.population, 0)
9	):: NUMERIC ,
10	2
11	) AS infected_population_percentage
12	FROM global_covid_stats g
13	JOIN country c
14	ON g.country_id = c.country_id
15	GROUP BY
16	c.name,
17	c.population
18	ORDER BY infected_population_percentage DESC ;

	country character varying (100) 🔒	population bigint 🔒	confirmed_cases bigint 🔒	infected_population_percentage numeric 🔒
1	Colombia	50882891	1272072	2.50
2	United States	331002651	8275066	2.50
3	Turkey	84339067	2108476	2.50
4	Nigeria	206139589	5153489	2.50
5	Spain	46754778	1168869	2.50
6	Argentina	45195774	1129894	2.50
7	Indonesia	273523615	6838090	2.50
8	Italy	60461826	1511545	2.50
9	Egypt	102334404	2558360	2.50

23. To find out the countries with the highest infection rates

Query	Query History
1	SELECT
2	c.name AS country,
3	c.population,
4	MAX(g.confirmed) AS confirmed_cases,
5	ROUND(
6	(
7	MAX(g.confirmed) * 100.0
8	/ NULLIF(c.population, 0)
9	)::NUMERIC,
10	2
11	) AS infection_rate
12	FROM global_covid_stats g
13	JOIN country c
14	ON g.country_id = c.country_id
15	GROUP BY
16	c.name,
17	c.population
18	ORDER BY infection_rate DESC ;

	country character varying (100) 🔒	population bigint 🔒	confirmed_cases bigint 🔒	infection_rate numeric 🔒
1	Colombia	50882891	1272072	2.50
2	United States	331002651	8275066	2.50
3	Turkey	84339067	2108476	2.50
4	Nigeria	206139589	5153489	2.50
5	Spain	46754778	1168869	2.50
6	Argentina	45195774	1129894	2.50
7	Indonesia	273523615	6838090	2.50
8	Italy	60461826	1511545	2.50
9	Egypt	102334404	2558360	2.50

24. To find out the countries and continents with the highest death counts

Countries

Query	Query History		country character varying (100)	continent character varying (50)	total_deaths bigint
1	SELECT				
2	c.name AS country,	1	India	Asia	621001
3	c.continent,	2	United States	North America	148951
4	MAX(g.deaths) AS total_deaths	3	Indonesia	Asia	123085
5	FROM global_covid_stats g	4	Brazil	South America	95651
6	JOIN country c	5	Nigeria	Africa	92762
7	ON g.country_id = c.country_id	6	Russia	Europe	65670
8	GROUP BY	7	Mexico	North America	58019
9	c.name,	8	Japan	Asia	56914
10	c.continent	9	Egypt	Africa	46050
11	ORDER BY total_deaths DESC;				

Continents

Query	Query History		continent character varying (50)	total_deaths numeric
1	SELECT			
2	c.continent,	1	Asia	3229323
3	SUM(g.deaths) AS total_deaths	2	North America	783829
4	FROM global_covid_stats g	3	Europe	740378
5	JOIN country c	4	Africa	579248
6	ON g.country_id = c.country_id	5	South America	486094
7	GROUP BY c.continent	6	Oceania	40157
8	ORDER BY total_deaths DESC;			

25. Average number of deaths by day (Continents and Countries) [Hint: order by , group by clause]

By Country

Query	Query History		country character varying (100)	average_daily_deaths numeric
1	SELECT	1	India	93150.00000000000000
2	c.name AS country,	2	United States	22342.00000000000000
3	AVG(g.new_deaths) AS average_daily_deaths	3	Indonesia	18462.00000000000000
4	FROM global_covid_stats g	4	Brazil	14347.33333333333333
5	JOIN country c	5	Nigeria	13914.00000000000000
6	ON g.country_id = c.country_id	6	Russia	9850.0000000000000000
7	GROUP BY c.name	7	Mexico	8702.3333333333333333
8	ORDER BY average_daily_deaths DESC;	8	Japan	8537.0000000000000000
		9	Egypt	6907.0000000000000000

By Continent

Query	Query History		continent character varying (50)	average_daily_deaths numeric
1	SELECT			
2	c.continent,	1	Asia	31460.333333333333
3	AVG(g.new_deaths) AS average_daily_deaths	2	North America	11197.111111111111
4	FROM global_covid_stats g	3	Africa	8274.66666666666667
5	JOIN country c	4	South America	6944.00000000000000
6	ON g.country_id = c.country_id	5	Europe	5287.94444444444444
7	GROUP BY c.continent	6	Oceania	1721.00000000000000
8	ORDER BY average_daily_deaths DESC ;			

26. Average of cases divided by the number of population of each country (TOP 10) [Hint: Limit]

Query	Query History		country character varying (100)	population bigint	average_cases numeric	cases_population_percentage numeric
1	SELECT					
2	c.name AS country,					
3	c.population,					
4	AVG(g.confirmed) AS average_cases,					
5	ROUND(					
6	(					
7	AVG(g.confirmed) * 100.0					
8	/ NULLIF(c.population, 0)					
9	)::NUMERIC,					
10	2					
11	) AS cases_population_percentage					
12	FROM global_covid_stats g					
13	JOIN country c					
14	ON g.country_id = c.country_id					
15	GROUP BY					
16	c.name,					
17	c.population					
18	ORDER BY cases_population_percentage DESC					
19	LIMIT 10;					

	country character varying (100)	population bigint	average_cases numeric	cases_population_percentage numeric
1	Colombia	50882891	742042.000000000000	1.46
2	United States	331002651	4827121.500000000000	1.46
3	Turkey	84339067	1229944.000000000000	1.46
4	Nigeria	206139589	3006201.500000000000	1.46
5	Spain	46754778	681840.000000000000	1.46
6	Argentina	45195774	659104.500000000000	1.46
7	Indonesia	273523615	3988885.500000000000	1.46
8	Italy	60461826	881734.166666666667	1.46
9	Egypt	102334404	1492376.333333333333	1.46

27. Considering the highest value of total cases, which countries have the highest rate of infection in relation to population? [Hint: Where clause]

Query	Query History		country character varying (100)	population bigint	total_cases bigint	infection_rate numeric
1	SELECT					
2	c.name AS country,					
3	c.population,	1	Colombia	50882891	1272072	2.50
4	MAX(g.confirmed) AS total_cases,	2	United States	331002651	8275066	2.50
5	ROUND(					
6	(					
7	MAX(g.confirmed) * 100.0	3	Turkey	84339067	2108476	2.50
8	/ NULLIF(c.population, 0)	4	Nigeria	206139589	5153489	2.50
9	)::NUMERIC,					
10	2					
11	) AS infection_rate	5	Spain	46754778	1168869	2.50
12	FROM global_covid_stats g	6	Argentina	45195774	1129894	2.50
13	JOIN country c					
14	ON g.country_id = c.country_id	7	Indonesia	273523615	6838090	2.50
15	GROUP BY					
16	c.name,	8	Italy	60461826	1511545	2.50
17	c.population					
18	HAVING MAX(g.confirmed) > 0	9	Egypt	102334404	2558360	2.50
19	ORDER BY infection_rate DESC;					

28. Countries with the highest number of deaths

Query	Query History		country character varying (100)	total_deaths bigint
1	SELECT	1	India	621001
2	c.name AS country,	2	United States	148951
3	MAX(g.deaths) AS total_deaths	3	Indonesia	123085
4	FROM global_covid_stats g	4	Brazil	95651
5	JOIN country c	5	Nigeria	92762
6	ON g.country_id = c.country_id	6	Russia	65670
7	GROUP BY c.name	7	Mexico	58019
8	ORDER BY total_deaths DESC;	8	Japan	56914
		9	Egypt	46050

29. Continents with the highest number of deaths

Query	Query History		continent character varying (50)	total_deaths numeric
1	SELECT	1	Asia	3229323
2	c.continent,	2	North America	783829
3	SUM(g.deaths) AS total_deaths	3	Europe	740378
4	FROM global_covid_stats g	4	Africa	579248
5	JOIN country c	5	South America	486094
6	ON g.country_id = c.country_id	6	Oceania	40157
7	GROUP BY c.continent			
8	ORDER BY total_deaths DESC ;			

Queries on Vaccination

30. Total vaccinated with at least 1 dose over time (All countries)

Query	Query History
1	SELECT
2	v.date,
3	SUM(v.first_dose) AS total_people_vaccinated
4	FROM vaccination v
5	GROUP BY v.date
6	ORDER BY v.date;

	date date	total_people_vaccinated numeric
1	2021-01-...	48276
2	2021-01-...	58604
3	2021-01-...	99449
4	2021-01-...	195525
5	2021-01-...	251280
6	2021-01-...	365965
7	2021-01-...	549381
8	2021-01-...	759008
9	2021-01-...	835058

31. Percentage of the population vaccinated with at least the first dose until 30/9/2021 (Top 3)

Query	Query History
1	SELECT
2	c.name AS country,
3	c.population,
4	MAX(v.first_dose) AS people_vaccinated_first_dose,
5	ROUND(
6	(
7	MAX(v.first_dose) * 100.0
8	/ NULLIF(c.population, 0)
9	)::NUMERIC,
10	2
11	) AS vaccination_percentage
12	FROM vaccination v
13	JOIN state s
14	ON v.state_id = s.state_id
15	JOIN country c
16	ON s.country_id = c.country_id
17	WHERE v.date <= '2021-09-30'
18	GROUP BY
19	c.name,
20	c.population
21	ORDER BY vaccination_percentage DESC
22	LIMIT 3;

	country character varying (100) 🔒	population bigint 🔒	people_vaccinated_first_dose bigint 🔒	vaccination_percentage numeric 🔒
1	India	1380004385	45932488	3.33

Using JOINS to combine the covid_deaths and covid_vaccine tables

32. To find out the population vs the number of people vaccinated

Query	Query History
1	SELECT
2	c.name AS country,
3	c.population,
4	SUM(v.first_dose) AS people_vaccinated_first_dose,
5	SUM(v.second_dose) AS people_fully_vaccinated
6	FROM country c
7	JOIN state s
8	ON c.country_id = s.country_id
9	JOIN vaccination v
10	ON s.state_id = v.state_id
11	GROUP BY
12	c.name,
13	c.population
14	ORDER BY people_vaccinated_first_dose DESC;

	country character varying (100) 🔒	population bigint 🔒	people_vaccinated_first_dose numeric 🔒	people_fully_vaccinated numeric 🔒
1	India	1380004385	28243114213	6758167261

33. To find out the percentage of different vaccine taken by people in a country

```

Query History
1  SELECT
2      c.name AS country,
3      SUM(v.covaxin) AS covaxin,
4      SUM(v.covishield) AS covishield,
5      SUM(v.sputnik_v) AS sputnik_v,
6      ROUND(
7          (
8              SUM(v.covaxin) * 100.0
9              /
10             NULLIF(
11                 SUM(v.covaxin)
12                 + SUM(v.covishield)
13                 + SUM(v.sputnik_v),
14                 0
15             )
16         )::NUMERIC,
17         2
18     ) AS covaxin_percentage,
19     ROUND(
20         (
21             SUM(v.covishield) * 100.0
22             /
23             NULLIF(
24                 SUM(v.covaxin)
25                 + SUM(v.covishield)
26                 + SUM(v.sputnik_v),
27                 0
28             )
29         )::NUMERIC,
30         2
31     ) AS covishield_percentage,
32     ROUND(
33         (
34             SUM(v.sputnik_v) * 100.0
35             /
36             NULLIF(
37                 SUM(v.covaxin)
38                 + SUM(v.covishield)
39                 + SUM(v.sputnik_v),
40                 0
41             )
42         )::NUMERIC,
43         2
44     ) AS sputnik_v_percentage
45 FROM country c
46 JOIN state s
47     ON c.country_id = s.country_id
48 JOIN vaccination v
49     ON s.state_id = v.state_id
50 GROUP BY c.name
51 ORDER BY country;

```

	country character varying (100)	covaxin numeric	covishield numeric	sputnik_v numeric	covaxin_percentage numeric	covishield_percentage numeric	sputnik_v_percentage numeric
1	India	3980450783	30963371665	14459217	11.39	88.57	0.04

34. To find out percentage of people who took both the doses

Query	Query History
1	SELECT
2	c.name AS country,
3	c.population,
4	SUM(v.second_dose) AS people_fully_vaccinated,
5	ROUND(
6	(
7	SUM(v.second_dose) * 100.0
8	/ NULLIF(c.population, 0)
9	::NUMERIC,
10	2
11	) AS fully_vaccinated_percentage
12	FROM country c
13	JOIN state s
14	ON c.country_id = s.country_id
15	JOIN vaccination v
16	ON s.state_id = v.state_id
17	GROUP BY
18	c.name,
19	c.population
20	ORDER BY fully_vaccinated_percentage DESC;

	country character varying (100)	population bigint	people_fully_vaccinated numeric	fully_vaccinated_percentage numeric
1	India	1380004385	6758167261	489.72

35. Total State-wise Confirmed Cases

Query	Query History	state character varying (100)	total_confirmed_cases bigint
1	SELECT	1 Maharashtra	1127721063
2	s.name AS state,	2 Karnataka	488861464
3	SUM(cs.confirmed) AS total_confirmed_cases	3 Kerala	458906023
4	FROM covid_case_stats cs	4 Tamil Nadu	431928644
5	JOIN state s	5 Andhra Pradesh	392432753
6	ON cs.state_id = s.state_id	6 Uttar Pradesh	312625843
7	GROUP BY s.name	7 Delhi	287237133
8	ORDER BY total_confirmed_cases DESC;	8 West Bengal	263107876
		9 Chhattisgarh	163776262
		10 Rajasthan	162369656

36. Maximum Active cases State-wise till date

Query	Query History		state character varying (100)	maximum_active_cases integer
1	SELECT	1	Maharashtra	701614
2	s.name AS state,	2	Karnataka	605515
3	MAX(cs.active_cases) AS maximum_active_cases	3	Kerala	445692
4	FROM covid_case_stats cs	4	Tamil Nadu	313048
5	JOIN state s	5	Uttar Pradesh	310783
6	ON cs.state_id = s.state_id	6	Rajasthan	212753
7	GROUP BY s.name	7	Andhra Pradesh	211554
8	ORDER BY maximum_active_cases DESC;	8	Gujarat	148297
		9	West Bengal	132181
		10	Chhattisgarh	131245

37. Max Per Day Confirmed cases in States

Query	Query History		state character varying (100)	maximum_daily_confirmed_cases integer
1	SELECT	1	Maharashtra	68631
2	s.name AS state,	2	Karnataka	50112
3	MAX(cs.new_confirmed) AS maximum_daily_confirmed_cases	3	Kerala	43529
4	FROM covid_case_stats cs	4	Uttar Pradesh	37944
5	JOIN state s	5	Tamil Nadu	36184
6	ON cs.state_id = s.state_id	6	Delhi	28395
7	GROUP BY s.name	7	Andhra Pradesh	24171
8	ORDER BY maximum_daily_confirmed_cases DESC;	8	West Bengal	20846
		9	Rajasthan	18298
		10	Chhattisgarh	17397

38. Max Per Day Death cases in States

Query	Query History		state character varying (100)	maximum_daily_deaths integer
1	SELECT	1	Bihar	3971
2	s.name AS state,	2	Maharashtra	3656
3	MAX(cs.new_deaths) AS maximum_daily_deaths	3	Madhya Pradesh	1481
4	FROM covid_case_stats cs	4	Karnataka	624
5	JOIN state s	5	Tamil Nadu	518
6	ON cs.state_id = s.state_id	6	Delhi	448
7	GROUP BY s.name	7	Uttar Pradesh	372
8	ORDER BY maximum_daily_deaths DESC;	8	Chhattisgarh	279
		9	Punjab	231
		10	Kerala	227

39. State-wise Mortality Rate

Query	Query History	state	confirmed_cases	deaths	mortality_rate
		character varying (100)	bigint	bigint	numeric
1	SELECT				
2	s.name AS state,	1	Punjab	99956177	2785594
3	SUM(cs.confirmed) AS confirmed_cases,	2	Maharashtra	1127721063	23868185
4	SUM(cs.deaths) AS deaths,	3	Uttarakhand	53140414	986001
5	ROUND(	4	Delhi	287237133	4943294
6	(	5	Sikkim	3186799	53150
7	SUM(cs.deaths) * 100.0	6	Himachal Pradesh	30238394	494855
8	/ NULLIF(SUM(cs.confirmed), 0)	7	Goa	28240159	447801
9	:::NUMERIC,	8	Puducherry	20065891	312155
10	2	9	Gujarat	143420082	2219448
11	) AS mortality_rate	10	West Bengal	263107876	3846989
12	FROM covid_case_stats cs				
13	JOIN state s				
14	ON cs.state_id = s.state_id				
15	GROUP BY s.name				
16	ORDER BY mortality_rate DESC;				

40. Covid Waves in 'Mumbai'

Daily Mumbai COVID data

Query	Query History
1	SELECT
2	report_date,
3	SUM(new_confirmed) AS new_cases,
4	SUM(new_deaths) AS new_deaths,
5	SUM(active_cases) AS active_cases
6	FROM mumbai_covid_waves
7	GROUP BY report_date
8	ORDER BY report_date;

	report_date	new_cases	new_deaths	active_cases
	date	bigint	bigint	bigint
1	2020-03-09	0	0	0
2	2020-03-10	0	0	1
3	2020-03-11	0	0	0
4	2020-03-12	1	0	2
5	2020-03-13	0	0	3
6	2020-03-14	0	0	3
7	2020-03-15	3	0	7
8	2020-03-16	0	0	7
9	2020-03-17	1	0	8

Mumbai 7-day moving average

Query	Query History
1	WITH daily_cases AS (
2	SELECT
3	report_date,
4	SUM(new_confirmed) AS new_cases
5	FROM mumbai_covid_waves
6	GROUP BY report_date
7	),
8	moving_average AS (
9	SELECT
10	report_date,
11	new_cases,
12	ROUND(
13	AVG(new_cases) OVER (
14	ORDER BY report_date
15	ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
16	),
17	2
18	) AS seven_day_average
19	FROM daily_cases
20	)
21	SELECT
22	report_date,
23	new_cases,
24	seven_day_average
25	FROM moving_average
26	ORDER BY report_date;

	report_date date	new_cases bigint	seven_day_average numeric
1	2020-03-09	0	0.00
2	2020-03-10	0	0.00
3	2020-03-11	0	0.00
4	2020-03-12	1	0.25
5	2020-03-13	0	0.20
6	2020-03-14	0	0.17
7	2020-03-15	3	0.57
8	2020-03-16	0	0.57
9	2020-03-17	1	0.71